Instituto Tecnológico de Costa Rica

Escuela de Administración de Tecnologías de Información




# Laboratorio #3

## Base de Datos Distribuida

Conjuntos de réplicas en MongoDB: mayoría, elección y durabilidad

Bases de Datos Avanzados

Profesor:
Manuel Zumbado Corrales

Por:
Emmanuel Esquivel Chavarría — 2022312336
Yerik Chaves Serrano — 2022437493
Jose Pablo Guerrero Duarte — 2022099311

Noviembre, 2026

# 1. Configuración del conjunto de réplicas

El laboratorio implementó **MongoDB 8.0** sobre Docker Compose con tres procesos `mongod`, cada uno con un volumen persistente y la opción `-replSet rs-lab3`. La aplicación Python usa `pymongo` y se conecta al conjunto por los tres nombres internos. No hay puertos publicados al host ni latencia geográfica: todos los contenedores residen en una computadora. No se habilitó autenticación ni TLS; es un entorno de pruebas.

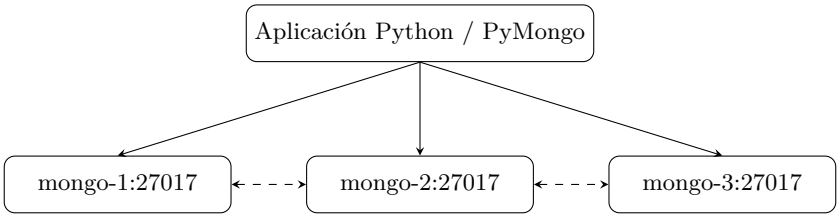



*Figura 1. Topología lógica; cada nodo mantiene su volumen MongoDB.*

### 1.1. Inicialización y significado de la configuración

```
rs.initiate({_id: "rs-lab3", members: [
{_id: 0, host: "mongo-1:27017"},
{_id: 1, host: "mongo-2:27017"},
{_id: 2, host: "mongo-3:27017"}]})
```

`_id` identifica el conjunto y debe coincidir con `-replSet`; `members` define los integrantes, su identificador numérico y su dirección `host` accesible mediante DNS de Compose. `localhost` sería incorrecto porque apuntaría al propio contenedor, no a los otros miembros. Tres votantes permiten mayoría de $\lfloor 3/2 \rfloor + 1 = 2$ y evitan empates entre dos nodos. Las opciones omitidas se completaron con los valores predeterminados.

### 1.2. Inspección, votación y replicación

| Parámetro observado | Valor e interpretación |
|---|---|
| `term` inicial | 1; aumenta al elegir un nuevo primario. |
| `heartbeatIntervalMillis` | 2 000 ms; frecuencia de latidos entre miembros. |
| `electionTimeoutMillis` | 10 000 ms; plazo de detección que influye en la elección, no cota completa de RTO. |
| `votes`, `priority` | 1 y 1 en los tres nodos: todos votan y son elegibles. |
| Estado inicial | `mongo-1` PRIMARY; `mongo-2` y `mongo-3` SECONDARY; retraso observado 0 s. |

*Cuadro 1. Inspección de `rs.status()` y `rs.conf()`; evidencia: `rs-inspect.txt`.*

El primario acepta escrituras y los secundarios reproducen su *oplog*. Su `syncSourceHost` fue inicialmente `mongo-1`; el origen puede cambiar. `db.hello()` permite identificar `isWritablePrimary`, `secondary` y `setName`. Un `priority` mayor favorece a un candidato elegible, pero no reemplaza el requisito de mayoría. El `term` es comparable al término de Raft del laboratorio anterior; MongoDB no utiliza Raft como protocolo de replicación. Las elecciones dependen de la detección de fallas y del voto mayoritario.

## 2. Datos, índices, oplog y lecturas

### 2.1. Generación y carga (TODO 1 y TODO 2)

La semilla reproducible **2026** generó **400 eventos**, **1 189 zonas** y una capacidad agregada de **162 810** entradas. El manifiesto guardó el SHA-256 `9fc6a2dbf450340038a53813d1bf85e9206da1f7f8460259bf8cf4609ad4774a`; el verificador finalizó **6/6**. `cargar_eventos()` lee JSONL e inserta por lotes: evita cargas individuales y acota el uso de memoria. Se utilizó `ordered=False` porque permite continuar con los documentos restantes del lote si uno falla, útil en una carga controlada con verificación posterior; no hace atómica la inserción de todo el lote.

Los índices compuestos elegidos fueron `eventos(categoria,fecha)` y `ordenes(evento_id,estado)`. El primero aprovecha igualdad en `categoria` y rango/ordenamiento por `fecha`; el segundo corresponde a la consulta de órdenes por evento y estado.

| Plan | Devueltos | Docs. examinados | Keys | Tiempo | Etapas |
|---|---|---|---|---|---|
| Con índice | 67 | 67 | 67 | 2 ms | `IXSCAN`, sin `SORT` |
| Sin índice | 67 | 400 | 0 | 1 ms | `COLLSCAN`, `SORT` |

*Cuadro 2. `explain("executionStats"); explain-indices.txt`.*

El índice redujo **83,25 %** los documentos examinados (400 a 67). No se afirma que fuera más rápido con 400 registros: las mediciones fueron 2 frente a 1 ms, por lo que la mejora demostrada es de trabajo evitado, no de tiempo en este conjunto pequeño.

### 2.2. Oplog y retraso de replicación (paso 6)

Se insertó `ORD-MANUAL-1` en `boleteria.ordenes`; apareció en `local.oplog.rs` con `op:"i"`, `ns:"boleteria.ordenes"`, término `t:1` y operación `o` con el documento. `ts` marca su posición temporal lógica; `ns` indica la colección. La orden fue visible en un secundario, mientras un intento de escribir directamente en él devolvió `not primary`. `rs.printSecondaryReplicationInfo()` indicó retraso de **0 s** al medirlo, no replicación instantánea garantizada.

El oplog es una colección de tamaño limitado que conserva una ventana móvil de cambios. Se observó una ventana de **12 480 s** (3 h 28 min) en esa captura. Si un secundario se atrasa más que los cambios retenidos, puede requerir sincronización inicial para reincorporarse, pues ya no puede recuperar todo el historial solamente desde el oplog.

### 2.3. Lecturas obsoletas en secundario (paso 7)

| Modo | Intentos | Obsoletas | Porcentaje |
|---|---|---|---|
| `w1_secundario` | 500 | 493 | 98,60 % |
| `majority_secundario` | 500 | 0 | 0,00 % |
| `causal_majority` | 500 | 0 | 0,00 % |

*Cuadro 3. Resultado de `lectura-secundario.csv`.*

**Hipótesis de trabajo:** `w:1` permitiría más lecturas atrasadas; `majority` las reduciría, y una sesión causalmente consistente con lectura de mayoría evitaría leer un estado anterior a la escritura propia. Se observó 98,60 % frente a 0 % y 0 %. El 0 % de `majority_secundario` fue *empírico*: dos de tres miembros pueden confirmar sin que el secundario elegido haya aplicado aún el cambio. La causalidad sí resuelve la exigencia de «leer mi escritura», con la configuración adecuada.

Para *mis entradas compradas* usaríamos lecturas primarias o sesión causal con `readConcern: majority`; para *eventos destacados*, un secundario potencialmente atrasado puede ser aceptable. En términos CAP, la lectura débil privilegia disponibilidad y tolera obsolescencia, mientras las garantías fuertes restringen qué resultados son visibles; el experimento no constituye una partición de red real.

## 3. Atomicidad, idempotencia y costo de garantías

### 3.1. Compra concurrente (TODO 3)

La implementación registra la orden usando su `_id` como clave de idempotencia; después ejecuta una actualización `update_one` sobre un único evento. El filtro usa `$elemMatch` para exigir que *la misma zona* cumpla nombre y disponibilidad suficiente; el incremento `$inc` descuenta entradas del elemento correspondiente. MongoDB reevalúa el filtro cuando compiten actualizaciones al mismo documento: no se pueden vender entradas por encima de la disponibilidad mediante esta operación.

En la prueba de 200 compradores y 120 entradas se confirmaron **120**, se rechazaron **80** y quedaron **0** disponibles. También pasaron compras múltiples, cinco reintentos de una orden (sin descuento duplicado) y una zona inexistente. Las seis verificaciones finalizaron correctamente. La versión deliberadamente incorrecta (lectura, decisión en Python y escritura no condicionada) confirmó **188** compras y dejó **–68** entradas: *read–modify–write* no es una operación atómica. Evidencias: `comprar.txt` y `comprar-incorrecto.txt`.

**Por qué `$elemMatch`:** si una zona A tiene nombre VIP pero saldo insuficiente y otra zona B tiene saldo suficiente, dos condiciones separadas sobre `zonas.nombre` y `zonas.disponibles` podrían coincidir en elementos distintos. `$elemMatch` obliga a que las dos se cumplan en una misma zona. La atomicidad de `update_one` protege un solo documento, *no* el par orden–evento. Si el proceso muere después del descuento pero antes de marcar la orden como confirmada, puede quedar una orden pendiente con inventario rebajado. Se requiere conciliación con identificadores únicos, transacción multidocumento o una operación recuperable explícita; el esquema actual no proporciona exactamente una vez de extremo a extremo.

### 3.2. *Write concern* (TODO 4)

Se instrumentó `insert_one` con `time.perf_counter()`, una colección configurada con `WriteConcern` para cada variante, **30 escrituras de calentamiento descartadas** y **300 muestras medidas** por variante (1 200 muestras, más cabecera). Percentiles por rango más cercano.

| Garantía | $n$ | p50 ms | p95 ms | p99 ms | ops/s |
|---|---|---|---|---|---|
| `w:1` | 300 | 0,57 | 1,32 | 2,59 | 1 404 |
| `w:1,j:true` | 300 | 6,35 | 12,80 | 20,49 | 137 |
| `w:majority` | 300 | 6,71 | 11,50 | 30,84 | 127 |
| `w:3` | 300 | 1,36 | 1,90 | 3,59 | 680 |

*Cuadro 4. Evidencias: `write-concern.txt` y 1 200 filas de `write-concern.csv`.*

**Predicción de referencia:** `w:1` sería el más rápido; `j:true` y `majority` elevarían el costo; `w:3` podría ser el más exigente. **Observado:** `majority` tuvo un p50 **11,8 veces** superior a `w:1`. Sorprendentemente, `w:3` fue más rápido que `majority` en esta corrida: se reporta como resultado local, no como propiedad general. Con tres miembros, `majority` requiere dos confirmaciones y `w:3` las tres; `j:true` añade persistencia en el diario. En redes regionales, esperar réplicas suele costar más por el RTT; `w:1` depende menos de esa distancia. **3.3. Entorno de ejecución y límites**

Entorno de trabajo: equipo con procesador Intel Core i7-1165G7, **16 GB de RAM** y Windows de 64 bits con Docker; cada `mongod` usa `-wiredTigerCacheSizeGB 0.25`. Los datos de hardware proceden de la configuración reportada para el equipo y deben contrastarse con la máquina que ejecutó todas las mediciones antes de entregar. La latencia entre contenedores es local y comparte CPU, almacenamiento y memoria; no representa tres máquinas independientes ni tres regiones.

## 4. Fallas A y B: disponibilidad y elección

Las sondas insertaron un documento con secuencia cada **0,5 s**, usaron `w:majority`, `wtimeout=2000 ms` y escrituras reintentables. Se guardaron CSV con tiempos de operación, marcas de caída y verificaciones de RPO con `readConcern:majority`. Se siguió el protocolo *precondición → evento → medición → verificación* y se recuperaron los contenedores antes de la siguiente prueba. El RPO se define solo sobre operaciones **confirmadas**, nunca sobre los intentos con error.

### 4.1. Línea base (paso 10)

El TODO 5, `calcular_rpo()`, compara conjuntos de secuencias confirmadas, con error y presentes. Los tres casos sintéticos pasaron. En la corrida `r45dc2f28` se registraron **20 intentos, 20 confirmados, 0 errores y 20 presentes**; **RPO = 0**. Esto fijó la referencia sin fallas.

### 4.2. Falla A: se detiene un secundario (paso 11)

**Predicción:** al detener `mongo-2`, `mongo-1` y `mongo-3` conservarían mayoría 2/3, sin interrupción de confirmaciones; quizá cambiarían algunas latencias. El 1 de noviembre de 2026, a las **22:50:06,486 UTC** (16:50:06 en Costa Rica), se detuvo `mongo-2`. Docker indicó `Running=false`; MongoDB lo reportó no saludable (`health=0`). Los otros dos miembros siguieron disponibles y `mongo-1` permaneció PRIMARY. Tras reiniciar `mongo-2`, los tres quedaron saludables.

| Medición | Antes | Después de señal |
|---|---|---|
| Escrituras confirmadas | 19 | 61 |
| p50 (ms) | 19,6 | 11,0 |
| p99 (ms) | 49,6 | 57,7 |

*Cuadro 5. Falla A, corrida `r81fe877b`; `falla-a.csv`.*

Hubo **80/80 confirmaciones**, **0 errores**, **0 pérdidas**. La primera confirmación posterior a la señal llegó a los 470 ms, pero no se observó suspensión del servicio: **no corresponde atribuir 470 ms a un RTO**. Al reincorporarse, el secundario reanudó la replicación del oplog pendiente desde una fuente válida; no se midió un atraso residual específico.

### 4.3. Falla B: se detiene el primario (paso 12)

**Predicción:** habría errores mientras se detectaba la pérdida de `mongo-1`, ocurría la elección y el driver descubría al nuevo PRIMARY; el RTO sería cercano o mayor a los 10 s de `electionTimeoutMillis`. Inicialmente el término era **1**; luego de `docker kill ti4601-mongo-1`, `mongo-3` resultó PRIMARY, `mongo-2` SECONDARY y el término aumentó a **2**. Al regresar, `mongo-1` quedó SECONDARY, pues el primario electo ya conservaba el respaldo mayoritario.

| Evento (UTC, 01/11/2026) | Evidencia |
|---|---|
| Última confirmación, `seq=10` | 23:01:45,469 |
| Caída del primario | 23:01:46,109 |
| Errores `seq=11,12,13` | `ServerSelectionTimeoutError` |
| Primera confirmación posterior, `seq=14` | 23:01:56,436 |

*Cuadro 6. Filas y marca temporal de `falla-b.csv` / `falla-b-stop.epoch`.*

$$\mathrm{RTO}_B = 1793574116.435820 - 1793574106.109 \approx \boxed{10.327\ \mathrm{s}}.$$

La ventana entre la última confirmación anterior y la primera posterior fue **10,967 s**; incluye el intervalo previo a la detención. Se registraron **101 intentos, 98 confirmados, 3 errores**, todos del tipo `ServerSelectionTimeoutError`, y **0 escrituras confirmadas perdidas**. El RTO puede superar el `electionTimeoutMillis` por detección, votación, actualización de topología del driver y reintentos. `retryWrites` no elimina todos los errores cuando no hay primario seleccionable antes del plazo del cliente.

## 5. Falla C: pérdida de mayoría y resultados inciertos

### 5.1. Escenario y predicción

Se detuvieron los dos secundarios `mongo-1` y `mongo-2`, dejando en ejecución el antiguo primario `mongo-3`. **Predicción:** con 1 de 3 votos el conjunto no conservaría un primario elegible; durante un breve periodo podrían aplicarse cambios localmente, pero `w:majority` no podría confirmarlos. Los errores de tiempo de espera no implicarían necesariamente ausencia del documento.

A las **23:10:21,517 UTC** (17:10:21 en Costa Rica) ambos contenedores se detuvieron y Docker confirmó `false` para los dos. La consulta a MongoDB mostró ambos inaccesibles (`health=0`) y `mongo-3` como **SECONDARY**: el sobreviviente se degradó al detectar que no mantenía mayoría. Al restaurar los nodos, `mongo-3` regresó como PRIMARY, con `term` incrementado de **2 a 3**. La captura de estado no mide el instante exacto de degradación, solo demuestra que ya había ocurrido cuando se consultó.

### 5.2. Errores y RPO verificado

| Clasificación | Cantidad | Secuencias / interpretación |
|---|---|---|
| Confirmadas (`ok`) | 94 | Presentes tras recuperación |
| `WTimeoutError` | 4 | 12–15, presentes |
| `WriteConcernError` | 1 | 16, presente |
| `ServerSelectionTimeoutError` | 9 | 17–25, ausentes |
| Total de intentos | 108 | 99 documentos encontrados |

*Cuadro 7. Corrida `rd1cf5314`; `falla-c.txt`, `falla-c-rpo.txt`.*

De los **14 errores**, **5 sí quedaron escritos** (`seq=12-16`) y **9 no se encontraron**. El primer grupo pudo aplicarse localmente antes de perder la elegibilidad y replicarse después del restablecimiento, pese a que la confirmación requerida expiró. El segundo grupo no encontró primario disponible dentro del plazo del driver. El verificador identificó **94 confirmados presentes**, **0 confirmados perdidos** y ningún documento inesperado: **RPO observado = 0**. La persistencia de un error no debe confundirse con garantía de confirmación en el momento de su respuesta.

### 5.3. Tiempo de recuperación y latencias

| Evento de sonda (UTC) | Registro |
|---|---|
| Última confirmación previa, `seq=11` | 23:10:21,044 |
| Marca de la falla | 23:10:21,517 |
| Primer timeout, `seq=12` | 23:10:23,570 |
| Primera confirmación posterior, `seq=26` | 23:11:04,277 |

*Cuadro 8. Filas de `falla-c.csv` y marca `falla-c-stop.epoch`.*

$$\mathrm{RTO}_C \approx 1793574664.277 - 1793574621.517 = \boxed{42.760\ \mathrm{s}}.$$

La ventana entre la última y la primera confirmación fue **43,233 s**. De las 94 confirmadas, 12 ocurrieron antes de la falla y 82 después; el p50 pasó de **10,7** a **14,3 ms** y el p99 de **42,7** a **296,9 ms**. Los percentiles excluyen los intentos fallidos, cuyos tiempos incluyeron esperas de 2–4,6 s. Se mantuvo el aislamiento aproximadamente 35 s antes de restaurar los secundarios; el tiempo medido desde el cliente incluye la recuperación posterior.

### 5.4. Consecuencia para una boletería

En una compra real, devolver «error» a quien pagó puede conducir a repetir una operación *que sí se aplicó*. Por eso el servicio debe reconsultar el identificador único de orden y diseñar reintentos idempotentes. La ausencia de confirmación debe tratarse como resultado incierto, no como autorización para emitir otra compra diferente. Esta lección conecta la Falla C con la reserva atómica de entradas del TODO 3.

# 6. Interpretación, predicciones y conclusiones

## 6.1. Cuadro comparativo de escenarios

| Escenario | OK | Error | RPO | RTO | Evidencia principal |
|---|---|---|---|---|---|
| Base | 20 | 0 | 0 | — | Sin falla |
| A: secundario | 80 | 0 | 0 | No hubo corte | Mayoría 2/3 se conserva |
| B: primario | 98 | 3 | 0 | 10,327 s | Elección: término 1 a 2 |
| C: sin mayoría | 94 | 14 | 0 | 42,760 s | 5 errores quedaron escritos |

*Cuadro 9. Resumen de sondas y `verificar_rpo.py`; RTO desde marca de caída hasta primera confirmación posterior.*

Las predicciones de trabajo fueron compatibles con los tres escenarios: continuidad con un secundario detenido, interrupción y nueva elección al caer el primario, e imposibilidad de confirmar en ausencia de mayoría. La predicción más importante se corroboró en C: **error no implica documento ausente**. Las predicciones se presentan aquí como hipótesis técnicas contrastadas con el experimento; los archivos de evidencia no acreditan por sí solos el momento en que fueron redactadas.

## 6.2. CAP y PACELC

**CAP.** Ante una partición de red genuina, preservar consistencia de las escrituras confirmadas exige dejar de aceptar confirmaciones si no hay mayoría. La Falla C reprodujo la *pérdida de quórum por detención de procesos*, no una partición de red; ilustra la misma restricción de mayoría sin pretender medir comportamiento de una red particionada. La Falla A mantuvo servicio por disponibilidad de dos votantes, no por «escrituras sin consenso». Las lecturas dirigidas a secundarios con garantías débiles pueden ser obsoletas incluso sin fallas, como mostró el 98,60 % de `w:1`.

**PACELC.** Incluso cuando no hay falla (*else*), se paga un compromiso entre latencia y garantías de consistencia/durabilidad: `w:1` registró p50 de 0,57 ms frente a 6,71 ms de `majority`, mientras las lecturas causales con mayoría ofrecen una mejor garantía de «leer mi escritura». Durante pérdida de quórum, el sistema prefiere no confirmar a inventar una mayoría. MongoDB permite seleccionar garantías por operación; no corresponde asignar una etiqueta CAP/PACELC única a todos los modos y lecturas.

## 6.3. Respuestas operativas y límites

- **Durabilidad y elecciones.** `w:1` acepta el reconocimiento del primario, `j:true` exige diario, `majority` espera una mayoría y `w:3` a los tres. El nuevo PRIMARY se escoge por votación y el antiguo no lo desplaza automáticamente al volver.
- **Secundario retrasado.** Recupera entradas del oplog mientras estén retenidas; si sobrepasa su ventana, necesita resincronizar. El retraso 0 s es solo una observación puntual.
- **Seguridad de negocio.** Una sola actualización de evento con condición atómica evita sobreventa, pero no asegura atomicidad entre `ordenes` y `eventos`. Se necesita reconciliación o transacción multidocumento para atender una interrupción entre ambos pasos.
- **Validez externa.** Los tres nodos comparten máquina, recursos y red local. Las cifras de latencia y RTO dependen de Docker, carga del host, sistema operativo y temporizadores; no se extrapolan a regiones separadas. El *rollback* de escrituras sin confirmar no se forzó ni se demostró.

## 6.4. Trazabilidad y entrega

Los scripts `cargar.py` (TODO 1–2), `comprar.py` (TODO 3), `bench_write_concern.py` (TODO 4) y `verificar_rpo.py` (TODO 5) quedan versionados en `labs/lab3-mongo-replicas/`. La carpeta `evidence/` incluye `comandos.md`, `cluster-start.txt`, `rs-inspect.txt`, `datos-manifest.txt`, `config-check.txt`, `explain-indices.txt`, `oplog.txt`, pares CSV/TXT de lecturas y *write concern*, compras y línea base, además de `falla-a*`, `falla-b*` y `falla-c*`. Cada cifra tabulada aquí está asociada a su archivo crudo; las capturas de estado documentan las precondiciones y la recuperación.

## 6.5. Conclusión

El conjunto confirmó la tolerancia a la caída de un secundario sin perder disponibilidad de escritura, una elección automática tras perder el primario (RTO 10,327 s) y la detención de confirmaciones cuando se perdió la mayoría (42,760 s hasta la primera confirmación posterior). El **RPO fue cero** sobre todas las operaciones confirmadas en las cuatro sondas. La distinción entre operaciones *confirmadas*, *fallidas* y *con resultado incierto* es esencial para mantener compras idempotentes y explicar correctamente las garantías de MongoDB.